

PROGRAMMING-2

Final Project

Powerpuff Girls Game

Nada Hassan Abdelsalam ID: 8095

Jana Amr Fouad ID: 7973

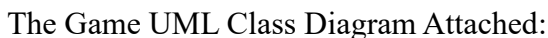
Hania Amr Taha ID: 8103

Nada Mohamed Abdelkader Allam ID: 8075

Dr. Laila Abo Hadid

The **Powerpuff Girls Game** is a single player-game in which the Professor catches falling characters: Blossom, Bubbles, Buttercup. The Professor must catch the same character three consecutive times to increase his score. In the Medium Level, Mojo Jojo is added to the falling characters. If the Professor catches Mojo Jojo his score decreases therefore he must avoid that. In the Hard Level, Satan is added to the falling characters in addition to Mojo Jojo. Catching Satan leads the Professor to lose a life. The Professor has three lives, if he loses all three lives the game is lost.

The following is the UML Class Diagram of the Java code of the game and the design patterns used in it:



Game UML (1).pdf

In the game the following design patterns are used:

- Singleton
- Factory
- Builder
- Iterator
- Strategy

1. SINGLETON

Singleton is a creational design pattern that allows you to guarantee that a class has just one instance while offering a global point of access to this instance.

Two issues are resolved by the Singleton pattern:

1. Verify that a class only contains one instance.
2. Give that instance a global access point.

In the game, a Singleton class is created that is used by PowerpuffWorld when creating the ObjectFactory which has one instance.

Singleton Class contains one attribute:

private static ObjectFactory instance;

Singleton Class contains one method:

- getInstance()

the getInstance() method returns the attribute instance if it is not null and creates a new ObjectFactory if it is null.

How do we use the Singleton Class?

In the Class called 'PowerpuffWorld' , we call the getInstance() method in the class only once when creating the ObjectFactory.

2. FACTORY

The Factory Design Pattern is a creational design pattern that provides an interface for creating objects in a super class, but allows subclasses to alter the type of objects that will be created. In other words, it defines an interface for creating an object, but it is the responsibility of the derived class or factory class to implement the method and decide which class to instantiate.

Factory Class contains no attributes and only one method: getObject() which takes a String as a parameter and returns a creation of a new object from the subclasses according to the String sent which represents a word which differentiates between each subclass. Ex: if a String “Bubbles” is sent to the method, a new object of Bubbles is created and returned to the caller Class.

How do we use the Factory Class?

In the Class called ‘PowerpuffWorld’ , we make an object from ObjectFactory class by using the Singleton design pattern as mentioned above. Then the getObject() in the ObjectFactory object is called each time a character in the game is created.

3. BUILDER

A Builder class provides a way to build the object step-by-step and provide a method that will actually return the final Object.

Builder Class contains 3 attributes:

List<GameObject> constant;

List<GameObject> moving;

List<GameObject> control;

Builder Class contains 6 methods:

- buildConstant()

- buildMoving()

- buildControl()

- getConstant()

- getMoving()

- getControl()

1. buildConstant() method:

this method adds the constant objects (background, hearts shape) to the list of constants.

2. buildMoving() method:

this method adds the moving objects (the falling objects) to the list of moving objects depending on the level of the game.

3. buildControl() method:

this method adds the controllable objects (the object that catches the falling objects) to the list of controllable objects.

4. `getConstant()`:

getter for the list of constants.

5. `getMoving()`:

getter for the list of moving.

6. `getControl()`:

getter for the list of control.

How do we use the Builder Class?

In the Class called 'PowerpuffWorld' , we make an object from Builder class. Then we invoke each method inside PowerpuffWorld class when we need to create or build any of the lists.

Every time we need to add or remove from any list, we will simply invoke any getter which returns the needed list to add/remove objects.

4. ITERATOR

The Iterator design pattern provides a way to access elements of a collection sequentially without exposing the underlying representation of the collection. It defines a separate object (the iterator) that encapsulates the behavior and keeps track of the current position within the collection.

The iterator allows iterating over a collection of GameObject instances

The key methods are:

- `hasNext()`: Checks if there are more elements in the collection.

- `next()`: Retrieves the next element in the collection.

In the context of the 'PowerpuffWorld' class, the iterator is used to iterate over the list of moving objects during the game loop. It allows the game logic to iterate over the moving objects without exposing the details of the underlying list implementation. This promotes a cleaner separation of concerns and makes the code more modular and maintainable.

5. STRATEGY

Strategy is a behavioral design pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable.

We use the Strategy when we have a lot of similar classes that only differ in the way they execute some behavior and this is the case in our game .The strategy design pattern is used to design the game levels (easy,medium,hard)

1. Interface "Strategy" is created which contains the following methods:

- `public int getSpeed();`

which controls the speed of the falling objects in the game, the hardest level has the maximum speed.

- `public int getTime();`

controls the time of each level

- `public int maxFallingGirls();`

maximum number of falling objects "powerPuff Girls"

- `public int maxFallingMojoJojos();`

maximum number of the villain which decreases the score "MOJOJOJO"

- `public int maxFallingSatans();`

maximum number of the villain which makes the player lose one life "SATAN"

2. #3 classes are created. Each class representing a game level.
3. Each concrete strategy class implements the Strategy interface and provides its own implementation for the methods, specifying the details of the strategy.

When a new game is created, The LevelsMenu class creates an instance of the selected concrete strategy and uses it to initialize the game. This way, the game behavior is determined by the selected strategy, and the client code. the Strategy Design Pattern in this code enables the LevelsMenu class to dynamically choose and use different game strategies without modifying its code, promoting flexibility and maintainability.